

Practical: 1

29/07/2026

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Algorithm:

Algorithms:

Bubble sort:

Step1: Start

Step2: Read n elements

Step3: compare adjacent elements

Step4: If $arr[j] > arr[j+1]$ swap them

Step5: Repeat for all elements in the current pass

Step6: Repeat the passes until the array is sorted

Step7: STOP

Selection Sort:

Step1: START

Step2: Read the number of elements n

Step3: Read the array $arr[0]$ to $arr[n-1]$

Step4: Set $i = 0$

Step5: Repeat while $i < n-1$:

- Set $minIndex = i$
- Set $j = i+1$
- Repeat while $j < n$:
 - If $arr[j] < arr[minIndex]$, set $minIndex = j$
 - Increment j
- Swap $arr[j]$ and $arr[minIndex]$
- Increment i

Step6: Display the sorted array

Step7: STOP

Insertion sort - Algorithm:

Step1: Start

Step2: Read the number of elements n

Step3: Read the array $arr[0]$ to $arr[n-1]$

Step4: Set $i = 1$

Step5: Repeat while $i < n$:

- Set $key = arr[i]$
- Set $j = i-1$
- Repeat while $j > 0$ and $arr[j] > key$:
 - Set $arr[j+1] = arr[j]$
 - Set $j = j-1$
- Set $arr[j+1] = key$

Step6: Increment i

Step7: Display the sorted array

Step8: STOP

Merge-sort - Algorithm:

Step1: START

Step2: Read the array

Step3: Find the middle element

Step4: Divide the array into two halves

Step5: Apply Merge-sort to the left half

Step6: Apply Merge-sort to the right half

Step7: Merge the two sorted halves

Step8: Display the sorted array

Step9: STOP

Quick-sort - Algorithm:

Step1: START

Step2: Read the array

Step3: Select an element as the pivot

Step4: Put smaller elements on the left of the pivot

Step5: Put larger elements on the right of the pivot

Step6: Apply Quick sort to the left part

Step7: Apply Quick sort to the right part

Step8: Display the sorted array

Step9: STOP

Program:

```
#include <iostream>
using namespace std;

// ----- BUBBLE SORT -----
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }
}

// ----- SELECTION SORT -----
void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;

        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }

        swap(arr[i], arr[minIndex]);
    }
}

// ----- INSERTION SORT -----
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
        }
    }
}
```

```
        j--;
    }

    arr[j + 1] = key;
}

// ----- MERGE SORT -----
void merge(int arr[], int low, int mid, int high)
{
    int temp[100];
    int i = low;
    int j = mid + 1;
    int k = low;

    while (i <= mid && j <= high)
    {
        if (arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= high)
        temp[k++] = arr[j++];

    for (i = low; i <= high; i++)
        arr[i] = temp[i];
}

void mergeSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);

        merge(arr, low, mid, high);
    }
}

// ----- QUICK SORT -----
```

```
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

// ----- DISPLAY -----
void display(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    cout << endl;
}

// ----- MAIN -----
int main()
{
    int n;
    int original[100];

    cout << "Enter number of elements: ";
    cin >> n;
```

```
cout << "Enter elements: ";
for (int i = 0; i < n; i++)
    cin >> original[i];

int bubble[100], selection[100], insertion[100];
int mergeArr[100], quick[100];

// Copy original array
for (int i = 0; i < n; i++)
{
    bubble[i] = original[i];
    selection[i] = original[i];
    insertion[i] = original[i];
    mergeArr[i] = original[i];
    quick[i] = original[i];
}

// Sorting
bubbleSort(bubble, n);
selectionSort(selection, n);
insertionSort(insertion, n);
mergeSort(mergeArr, 0, n - 1);
quickSort(quick, 0, n - 1);

// Display results
cout << "\nBubble Sort:  ";
display(bubble, n);

cout << "Selection Sort: ";
display(selection, n);

cout << "Insertion Sort: ";
display(insertion, n);

cout << "Merge Sort:  ";
display(mergeArr, n);

cout << "Quick Sort:  ";
display(quick, n);

return 0;
}
```

Output:

```
Enter number of elements: 6
Enter elements: 64 25 12 22 11 90

Bubble Sort:    11 12 22 25 64 90
Selection Sort: 11 12 22 25 64 90
Insertion Sort: 11 12 22 25 64 90
Merge Sort:     11 12 22 25 64 90
Quick Sort:     11 12 22 25 64 90
```

=== Code Execution Successful ===

Time Complexity:

Time complexity

1) Bubble-sort

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}
```

$T(n) = n(n-1) + n(n-1) + n(n-1) \Rightarrow 3n(n-1)$
 $T(n) = 3n(n-1) = T(n) = 3n^2 - 3n$
 Ignore constants and lower-order terms
 $T(n) = O(n^2)$

2) Selection-sort

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        int minIndex = i;
        for (int j = i+1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        swap(arr[i], arr[minIndex]);
    }
}
```

$T(n) = (n-1)(n-2)(n-3) + \dots + 1$
 $1+2+3+\dots+(n-1) = \frac{n(n-1)}{2}$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = \frac{n^2 - n}{2}$
 $T(n) = O(n^2)$

3) Insertion-sort

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i-1;
        while (j > 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
```

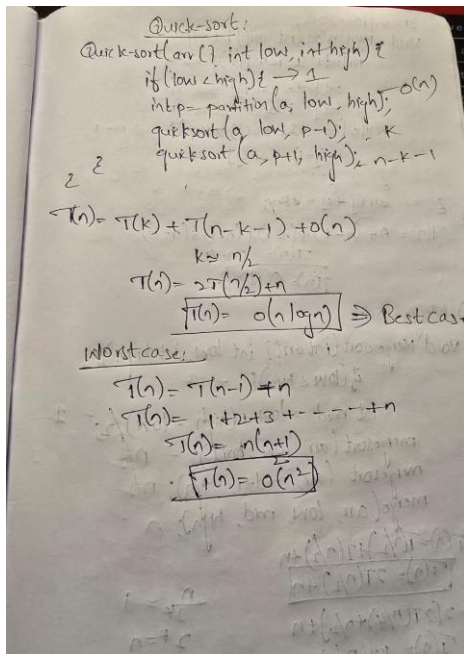
$T(n) = n + (n-1) + (n-1) + (n-1) + \dots + 1$
 $T(n) = \frac{n(n+1)}{2}$
 $T(n) = O(n^2)$

4) Merge-sort

```
void mergeSort(int arr[], int low, int high) {
    if (low < high) {
        int mid = low + (high - low) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid+1, high);
        merge(arr, low, mid, high);
    }
}
```

$T(n) = T(n/2) + T(n/2) + n$
 $T(n) = 2T(n/2) + n$
 $T(n) = 2[2T(n/4) + n/2] + n$
 $T(n) = 4T(n/4) + 2n$
 $T(n) = 8T(n/8) + 3n$
 $T(n) = 16T(n/16) + 4n$
 $T(n) = 2^k T(n/2^k) + kn$

$\frac{n}{2^k} = 1$
 $2^k = n$
 $k = \log_2 n$
 $T(n) = n + n + n + \dots$
 $T(n) = O(n \log n)$



Conclusion:

I understood that different sorting algorithms use different ways to arrange data. Bubble, Selection and Insertion Sort are easy to understand and implement, but they can take more time for large data. Merge Sort and Quick Sort use a divide-and-conquer approach and are generally more efficient for large datasets. Overall, choosing an algorithm depends on the size and condition of the data and the required